



FIGURE 5.11

A cut where cut events are not concurrent.

**TIME OF A CUT.** If each cut event is assigned a vector timestamp, then a cut in a distributed computation can be assigned a timestamp in the following manner:

If  $C = \{c_1, c_2, \dots, c_n\}$  is a cut, where  $c_i$  is a cut event at site  $S_i$  with vector timestamp  $VT_{c_i}$ , then the vector time of the cut,  $VT_c$ , is defined as follows:

$$VT_c = \sup(VT_{c_1}, VT_{c_2}, \dots, VT_{c_n})$$

where  $\sup$  is a componentwise maximum operation. That is,

$$VT_c[i] = \max(VT_{c_1}[i], VT_{c_2}[i], \dots, VT_{c_n}[i]).$$

**Theorem 5.2.** If  $C = \{c_1, c_2, \dots, c_n\}$  is a cut with a vector time  $VT_c$ , then the cut is consistent iff

$$VT_c = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$$

**Proof.** If  $C$  is a consistent cut, then from the previous theorem, events  $c_1, c_2, \dots, c_n$  are concurrent. In other words,  $\forall i \forall j, c_i[i] \geq c_j[i]$ . Therefore,

$$\sup(VT_{c_1}, VT_{c_2}, \dots, VT_{c_n}) = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$$

For sufficiency, note that  $VT_c = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$  implies that no message sent after a cut event  $c_i$  has been received before another cut event  $c_j$ . Thus, cut events  $c_1, c_2, \dots, c_n$  form a consistent cut.  $\square$

## 5.8 TERMINATION DETECTION

A distributed computation generally consists of a set of cooperating processes which communicate with each other by exchanging messages. In the case of a distributed computation, it is important to know when the computation has terminated. The problem of termination detection arises in many distributed algorithms and computations. For example, how to determine when an election, a deadlock detection, a deadlock resolution, or a token generating algorithm has terminated. Termination detection, in fact, is an example of the usage of the coherent view (consistent global state) of a distributed system. A large number of algorithms have been developed for termination detection. In this section, we present the termination detection algorithm proposed by Huang [8].

**SYSTEM MODEL.** A process may either be in an *active* state or *idle* state. Only active processes can send messages. An active process may become idle at any time. An idle process can become active on receiving a *computation* message. Computation messages are those that are related to the underlying computation being performed by

the cooperating processes. A computation is said to have terminated if and only if all the processes are idle and there are no messages in transit. The messages sent by the termination detection algorithm are referred to as *control* messages.

**BASIC IDEA.** One of the cooperating processes monitors the computation and is called the *controlling agent*. Initially all processes are idle, the controlling agent's weight equals 1, and the weight of the rest of the processes is zero. The computation starts when the controlling agent sends a computation message to one of the processes. Any time a process sends a message, the process's weight is split between itself and the process receiving the message (the message carries the weight for the receiving process). The weight received along with a message is added to the weight of the process. Thus, the algorithm assigns a weight  $W$  ( $0 < W \leq 1$ ) to each active process (including the controlling agent) and to each message in transit. The weights assigned are such that, at any time, they satisfy an invariant  $\sum W = 1$ . On finishing the computation, a process sends its weight to the controlling agent, which adds the received weight to its own weight. When the weight of the controlling agent is once again equal to 1, it concludes that the computation has terminated.

**NOTATIONS.** The following notations are used in the algorithm:

- $B(DW)$  = Computation message sent as a part of the computation and  $DW$  is the weight assigned to it.
- $C(DW)$  = Control message sent from the processes to the controlling agent and  $DW$  is the weight assigned to it.

### Huang's Termination Detection Algorithm

**Rule 1.** The controlling agent or an active process having weight  $W$  may send a computation message to a process  $P$  by doing:

Derive  $W_1$  and  $W_2$  such that  
 $W_1 + W_2 = W$ ,  $W_1 > 0$ ,  $W_2 > 0$ ;  
 $W := W_1$ ;  
 Send  $B(W_2)$  to  $P$ ;

**Rule 2.** On receipt of  $B(DW)$ , a process  $P$  having weight  $W$  does:

$W := W + DW$ ;  
 If  $P$  is idle,  $P$  becomes active;

**Rule 3.** An active process having weight  $W$  may become idle at any time by doing:

Send  $C(W)$  to the controlling agent;  
 $W := 0$ ;  
 (The process becomes idle);

**Rule 4.** On receiving  $C(DW)$ , the controlling agent having weight  $W$  takes the following actions:

$$W := W + DW;$$

If  $W = 1$ , conclude that the computation has terminated.

**PROOF OF CORRECTNESS.** Let

$A$ : The set of weights of all the active processes.

$B$ : The set of weights of all the computation messages in transit.

$C$ : The set of weights of all the control messages in transit.

$W_c$ : Weight of the controlling agent.

The following  $P_1$  and  $P_2$  are the invariants.

$$P_1: W_c + \sum_{W \in (A \cup B \cup C)} W = 1$$

$$P_2: \forall W \in (A \cup B \cup C), W > 0$$

Hence,

$$W_c = 1 \Rightarrow \sum_{W \in (A \cup B \cup C)} W = 0 \text{ by } P_1$$

$$\sum_{W \in (A \cup B \cup C)} W = 0 \Rightarrow (A \cup B \cup C) = \phi \text{ by } P_2$$

$$(A \cup B \cup C) = \phi \Rightarrow (A \cup B) = \phi$$

$(A \cup B) = \phi$  implies the termination of computation by definition. We can verify that the algorithm never detects a false termination as follows:

$$(A \cup B) = \phi \Rightarrow W_c + \sum_{W \in C} W = 1 \text{ by } P_1$$

Since the message transmission delay is finite, eventually  $W_c = 1$ . Therefore, the algorithm detects every true termination in finite time.

The above algorithm can be extended to dynamic systems where an active process may spawn off another active process or may migrate from one site to another. When one process creates another, its weight can be split as in Rule 1 and the two resulting weights can be assigned to the creating and the created processes.

An efficient scheme to implement the system of weights is presented in [8]. This scheme can generate a large number of weights while maintaining the invariants. The details of the implementation of the scheme of weights are beyond the scope of this book and the reader is referred to [8].